操作系统实验报告

实验基本信息

项目	内容
实验名称	Lab 3：中断与异常处理
小组成员	2413646-秦紫轩、2412933-李易桐、2413524-洪若曦
完成日期	2026-11-07

小组分工

成员	负责的模块
2412933-李易桐（A）	补全 `__alltraps` 与 `RESTORE_ALL` 以及其他入口配置
2413524-洪若曦（B）	实现 `interrupt_handler()`，完成时钟、软件、外部及未知中断的分派与相应处理。
2413646-秦紫轩（C）	实现 `exception_handler()`，处理非法指令、断点及其他异常；补齐累计输出后关机逻辑，完成报告撰写。

一、实验目的

本实验完成 RISC-V S 态中断与异常的处理，理解硬件记录 Trap 状态、汇编保存现场、C 函数分派处理以及恢复执行之间的关系。

主要目标如下：

1. 理解 `stvec`、`sstatus`、`sepc`、`scause`、`stval` 在 Trap 进入与返回过程中的作用。
2. 依据 `struct trapframe` 的布局补全汇编入口与现场恢复，使 C 处理函数能够接收并修改 Trap frame。
3. 实现持续时钟中断，维护 `ticks`，按规定间隔输出统计信息，并在累计输出达到配置阈值后关机。
4. 识别非法指令和断点异常，根据指令编码正确推进 `tf->epc`，避免返回后重复触发同一异常；对其他异常保留现场诊断输出。
5. 通过正常启动和现有评分脚本验证各模块整合后的行为。

二、实验环境

2.1 构建与运行环境

项目	配置
Linux 系统	Ubuntu 20.04.3 LTS
目标平台	QEMU `virt`，64 位 RISC-V

项目	配置
交叉编译器	`riscv64-unknown-elf-gcc` 10.2.0，SiFive GCC-Metal 10.2.0-2020.12.8
模拟器	QEMU 4.1.1
固件	OpenSBI v0.4，运行时 SBI 版本 0.1
构建与测试工具	GNU Make 4.2.1、Python 3.8.10、`tools/inject.py`、`tools/grade.sh`
内核加载方式	`-machine virt -nographic -bios default -kernel bin/ucore.img`

2.2 AI 工具

成员	AI 编程工具	底层模型	备注
2413646-秦紫轩	ChatGPT（VS Code 插件）	ChatGPT 6.1 Sol	C 模块、整合验证及报告整理
2412933-李易桐	ChatGPT（VS Code 插件）	ChatGPT 6.1 Sol	A 模块及启动兼容处理
2413524-洪若曦	ChatGPT（VS Code 插件）	ChatGPT 6.1 Sol	B 模块中断处理

三、实验整体逻辑分析

3.1 本章节的逻辑主线

本实验围绕"内核怎样响应事件并恢复执行"展开。初始化时先配置统一 Trap 入口，再设置时钟与中断使能；事件发生后，硬件记录原因和返回状态，汇编保存寄存器并调用 C 函数，C 层依据原因完成处理，最后恢复现场并执行 `sret`。

```
kern_init：清零 BSS、读取 DTB、初始化内存管理
    → idt_init：stvec = __alltraps
    → clock_init：设置第一次时钟事件
    → intr_enable：打开 S 态总中断开关
    ↓
发生交给 S 态处理的中断或异常
    ↓
CPU 记录 sepc / scause / stval，更新 sstatus，跳到 __alltraps
    ↓
SAVE_ALL 保存现场 → a0 = sp → 调用 trap(tf)【A 补全调用部分】
    ↓
trap_dispatch 根据 cause 最高位分派
    ├─ 中断 → interrupt_handler【B】
    │           → 时钟：设置下次事件、计数、输出
    │           → 累计输出达到阈值：sbi_shutdown【C 补齐】
    └─ 异常 → exception_handler【C】
                → 非法指令 / 断点：输出原地址、按长度推进 epc
                → 其他异常：打印 Trap frame
    ↓ 未关机且处理函数正常返回
__trapret → RESTORE_ALL【A】→ sret → 恢复执行
```

3.2 功能的逐步实现

1. 衔接启动与内存依赖：保留低地址恒等映射和高地址内核映射，设置精确栈顶，迁入物理内存初始化与 Best-Fit 实现，保证进入 Lab3 初始化流程。
2. 补全 Trap 入口与恢复：在已保存现场的基础上向 C 函数传递 `trapframe`，实现返回状态和通用寄存器的恢复。
3. 实现中断处理：持续设置时钟事件，每次处理中断增加 `ticks`，每 100 次输出一次统计；为其他中断提供提示或现场诊断。
4. 实现异常处理：区分非法指令、断点和其他异常，对需要跳过的指令按实际长度更新 `epc`。
5. 补齐关机并整体回归：按当前配置累计输出 10 次后关机，独立核对正常退出，再运行已有评分脚本，得到 `100/100`。

四、实验内容与实现

4.1 A：Trap 入口与现场恢复

负责人：2412933-李易桐。

模块功能描述

主要实现位于 trapentry.S，与 trap.h 中的 `struct trapframe` 配合：

```
__alltraps：汇编 Trap 入口，在 SAVE_ALL 后调用 trap(tf)
RESTORE_ALL：恢复返回状态、通用寄存器和原 sp 的汇编宏
__trapret：执行 RESTORE_ALL 和 sret 的统一返回位置
```

框架的 `SAVE_ALL` 将 32 个通用寄存器槽位及 4 项 CSR 状态保存到栈上，共占 RV64 下的 288 字节。A 在入口中执行 `mv a0, sp`，把 Trap frame 地址作为首个参数传给 `trap()`，再通过 `jal trap` 调用 C 层。

`RESTORE_ALL` 先读取 frame 中的 `sstatus`、`sepc` 并写回 CSR，再恢复通用寄存器，最后恢复 `sp`。将 `sp` 放在最后，是因为其余加载仍使用当前 frame 基址；提前恢复会使后续偏移定位错误。`x0` 恒为零，不需要恢复；`scause` 和 `stval` 用于判断与诊断，不作为返回位置写回。

最终提示词

归纳一下大概如下（不然篇幅太长），原始记录见 prompt_A.md。

```
[PROMPT]
补全 Lab3 的 __alltraps 调用部分与 RESTORE_ALL，衔接启动依赖。

[RELY]
读取 trapentry.S、trap.h、init.c、entry.S、Makefile 及内存管理代码。
已有 SAVE_ALL 和 struct trapframe，保持二者布局不变。

[GUARANTEE]
正确传递 Trap frame，恢复返回状态与可写通用寄存器，最后恢复 sp。
保留其他成员的中断、异常处理范围及原有测试判据。
```

```
[SPECIFICATION]
前置条件：sp 指向 SAVE_ALL 建立的 Trap frame。
后置条件：C 函数返回后恢复现场，由 sret 返回保存的执行位置。
验证要求：使用专项入口与寄存器探针检查，记录真实日志和验证范围。
```

4.2 B：中断分派

负责人：2413524-洪若曦。

模块功能描述

主要接口位于 trap.c，时钟接口由 clock.c 提供：

```
void interrupt_handler(struct trapframe *tf);
void clock_set_next_event(void);
extern volatile size_t ticks;
```

`trap_dispatch()` 已根据 `cause` 最高位识别中断，B 再用 `tf->cause & 0x7FFFFFFFFFFFFFFFULL` 提取编号并分支处理。

类型	当前处理
`IRQ_S_TIMER`，编号 5	设置下一次时钟事件，执行 `ticks++`；每 100 次调用 `print_ticks()`。
`IRQ_S_SOFT`，编号 1	输出 `Supervisor software interrupt`。
`IRQ_S_EXT`，编号 9	输出 `Supervisor external interrupt`。
其他中断	输出未知编号，再调用 `print_trapframe(tf)`。

`clock_set_next_event()` 使用 `sbi_set_timer(当前 time + 100000)` 设置下一次事件。每次处理中断都重新设置未来时刻，使内核持续获得定时事件；`ticks` 是已处理次数，与硬件 `time` 计数区分。

最终代码在每次统计输出后检查 `ticks / TICK_NUM >= SHUTDOWN_AFTER`。当前默认 `SHUTDOWN_AFTER=10`，即第 1000 次时钟中断输出第 10 行统计后调用 `sbi_shutdown()`。该默认值可通过编译宏覆盖。软件、外部中断分支目前仅提供提示，未实现完整设备驱动。

最终提示词

B 的原始记录见 prompt_B.md；关机补充任务见 prompt_C.md 中的“时钟中断的累计输出与关机”。

```
[PROMPT]
实现 interrupt_handler，处理中断分派、持续时钟及未知中断诊断。

[RELY]
使用 trapframe、IRQ_S_TIMER / IRQ_S_SOFT / IRQ_S_EXT、ticks、
clock_set_next_event、TICK_NUM、print_ticks 和 print_trapframe。
```

```
[GUARANTEE]
保持函数签名和统一 Trap 返回链路，不修改 A 或异常处理代码。

[SPECIFICATION]
时钟分支每次设置下次事件并增加一次 ticks，每 100 次输出统计。
软件和外部中断输出提示，未知中断打印编号和完整现场。
```

4.3 C：异常处理与执行位置更新

负责人：2413646-秦紫轩。

模块功能描述

主要接口位于 trap.c：

```
void exception_handler(struct trapframe *tf);
```

函数先根据 `tf->cause` 选择分支。非法指令输出 `Illegal instruction at 0x...`，断点输出 `Breakpoint at 0x...`，均报告修改前的异常地址。随后读取 `epc` 处的首个 16 位编码判断长度：

指令编码条件	返回位置更新
bits[1:0] 不等于 `11`	`tf->epc += 2`。
bits[1:0] 等于 `11`，且 bits[4:2] 不等于 `111`	`tf->epc += 4`。
更长编码	仅打印现场，不猜测恢复长度。

本实现恢复实验涉及的 16 位与 32 位指令，要求相应异常地址在当前内核映射中可读。其他异常在 `default` 中直接调用 `print_trapframe(tf)` 后返回，不读取 `epc` 指向的指令，也不修改 frame。

最终提示词

异常实现与关机补充记录见 prompt_C.md。

异常处理 Prompt

```
[PROMPT]
在 trap.c 中实现 exception_handler，输出非法指令和断点信息，
按指令长度更新 epc，其他异常调用已有现场打印函数。

[RELY]
读取 trap.c、trap.h、trapentry.S、riscv.h 和 defs.h。
CAUSE_ILLEGAL_INSTRUCTION=2，CAUSE_BREAKPOINT=3；
RESTORE_ALL 会将处理后的 tf->epc 写回 sepc。

[GUARANTEE]
保持签名和其他函数不变，已处理异常仅修改 epc，其他异常不修改 frame。
```

```
[SPECIFICATION]
从可读的 epc 地址读取首半字，识别 16 位或 32 位编码，分别推进 2 / 4 字节。
输出原异常地址，其他原因只打印现场，不解引用该原因携带的 epc。
完成普通编译、链接检查，功能验证在后续整合阶段进行。
```

时钟关机补充 Prompt

```
[PROMPT]
补齐时钟中断的累计输出与关机逻辑。仅修改 trap.c 的必要头文件、
关机阈值定义和 interrupt_handler 的时钟分支，保留其他实现。

[RELY]
已有 clock_set_next_event、ticks、TICK_NUM=100 和 print_ticks。
使用 libs/sbi.h 提供的 sbi_shutdown()。

[GUARANTEE]
新增可由编译宏覆盖的 SHUTDOWN_AFTER，表示累计统计输出次数。
当前采用默认值 10，即 1000 次时钟中断后关机。

[SPECIFICATION]
每次时钟中断仍先设置下次事件，再增加一次 ticks。
达到 TICK_NUM 的整数倍时先打印统计，累计输出达到阈值后调用关机接口。
编译后验证恰好输出 10 次统计并自动退出，再运行现有 make grade。
记录真实结果，不按评分宏改变功能行为，不修改测试判据或分值。
```

五、测试与验证

5.1 正常启动、持续时钟与关机

在 WSL 的工程目录中，可按以下命令复现正常启动：

```
cd /mnt/d/OS-Lab/projects/lab3/code
make
make qemu
```

图1显示初始化完成，随后连续输出 10 行 `100 ticks`，最终返回终端。正常启动验证的 QEMU 退出码为 0，说明当前默认配置下完成了持续时钟处理和累计输出后关机。

```
DTB Address: 0x82200000
Physical Memory from DTB:
  Base: 0x0000000080000000
  Size: 0x0000000008000000 (128 MB)
  End:  0x0000000087ffffff
DTB init completed
(THU.CST) os is loading ...
Special kernel symbols:
  entry  0xffffffffc0200054 (virtual)
  etext  0xffffffffc0202026 (virtual)
  edata  0xffffffffc0207028 (virtual)
  end    0xffffffffc0207498 (virtual)
Kernel executable memory footprint: 30KB
memory management: default_pmm_manager
Physical memory: [0x0000000080000000, 0x0000000088000000)
Free physical pages: 31928
check_alloc_page() succeeded!
satp virtual address: 0xffffffffc0206000
satp physical address: 0x0000000080206000
++ setup timer interrupts
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
100 ticks
(base) PS D:\OS-Lab\projects>
```

图1　正常启动与时钟关机结果。

5.2 整体回归评分

在相同工程目录执行：

```
make grade
```

```
Building re-armed supervisor timer and repeated trap return...
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/debug/panic.c
+ cc kern/debug/kdebug.c
+ cc kern/debug/kmonitor.c
+ cc kern/driver/dtb.c
+ cc kern/driver/clock.c
+ cc kern/driver/console.c
+ cc kern/driver/intr.c
+ cc kern/trap/trap.c
+ cc kern/trap/trapentry.S
+ cc kern/mm/pmm.c
+ cc kern/mm/default_pmm.c
+ cc kern/mm/best_fit_pmm.c
+ cc kern/test/test_lab3_context.S
+ cc kern/test/test_lab3.c
+ cc libs/string.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ ld bin/kernel
Running re-armed supervisor timer and repeated trap return...
  - re-armed supervisor timer and repeated trap return: PASS (40/40)
Total Score: 100/100
```

图2　评分末项通过，最终总分为 100/100。

六、实验总结与收获

6.1 对中断与异常机制的理解

1. Trap 处理需要硬件与软件共同完成。CPU 记录原因、返回地址和特权状态，入口汇编保存通用寄存器，C 函数完成处理，恢复宏与 `sret` 再将执行流接回。只有各阶段对现场布局和返回状态的约定一致，才能正确恢复运行。
2. 中断和异常共用入口，但处理后的执行位置不同。时钟中断通常保留原 `epc`；本实验的非法指令和断点需要按实际指令长度推进 `epc`。能打印异常信息不等于能够正确返回，还必须检查返回位置。
3. 持续时钟依赖每次处理时重新设置下一次事件，软件计数 `ticks` 则记录已处理次数。总使能位、类型使能位和待处理中断状态分别承担不同职责，配置入口与打开中断也需要区分。
4. 汇编调用 C 函数依靠调用约定和链接器解析符号。`jal trap` 的目标由 C 函数定义提供，链接器完成地址与跳转偏移的解析，无须在链接脚本中逐个定义函数。

6.2 AI 使用经验

AI 协作时，先在 Prompt 中明确依赖、修改范围和预期行为，再分阶段实现与验证。对 AI 的解释仍需结合真实源码、符号和反汇编核对；对运行结果则保留日志与截图。编译链接通过只说明工程能够生成，功能是否正确还需由实际执行与回归测试确认。

6.3 实验完成情况

到现在已完成 Trap 入口与现场恢复、中断分派、非法指令和断点异常处理，以及累计输出后关机。其他异常保留现场诊断，软件和外部中断保持实验要求的提示处理范围。